

LA Noire — Police Management System

A modern web application for managing police cases, investigations, crime scenes, complaints, evidence, suspects, and rewards. Built with React, TypeScript, and Vite, connecting to a Django REST API backend.

Table of Contents

- Responsibilities & Tasks
- Naming Conventions
- Commit Message Conventions
- Project Management
- CI/CD Flow
- Key Entities
- NPM Packages
- AI-Generated Code Samples
- AI Strengths & Weaknesses
- Requirements Analysis
- Getting Started

Responsibilities & Tasks

Member	Responsibilities	Tasks Completed
Moeen Aali	Architecture, structures, initial services, UI components, DevOps	Project architecture & structure, core services (HTTP client, auth store), UI component library (shadcn-based), initial pages and features, Docker, CI/CD, nginx config
Melika Alizadeh	Pages, functionality, bug fixes, access control	Additional pages and features, bug fixes, role-based access control (RoleGuard, route permissions)

Naming Conventions

- Files:** kebab-case for all files (e.g. `case-detail.page.tsx`, `auth.store.ts`).
- Components:** PascalCase for React components (e.g. `CaseCard`, `EvidenceNode`).
- Pages:** Suffix with `.page.tsx` (e.g. `home.page.tsx`, `login.page.tsx`).
- Stores:** Suffix with `.store.ts` (e.g. `auth.store.ts`, `detective-board.store.ts`).
- API modules:** Suffix with `.ts` in `src/api/` (e.g. `auth.ts`, `cases.ts`).
- Types:** Suffix with `.type.ts` (e.g. `case.type.ts`, `evidence.type.ts`).
- Schemas:** Suffix with `.schema.ts` for Zod validation (e.g. `auth.schema.ts`).
- Guards:** Suffix with `.guard.tsx` (e.g. `role.guard.tsx`).
- Variables/Functions:** camelCase (e.g. `getEmployeeCount`, `accessToken`).
- Constants:** UPPER_SNAKE_CASE or PascalCase for exported const arrays (e.g. `ALLOWED_CASE_ROLES`).
- Interfaces:** PascalCase, no `I` prefix (e.g. `Case`, `CaseDetail`, `CreateCaseRequest`).
- Route paths:** kebab-case (e.g. `/cases/:id`, `/crime-scenes/new`, `/reward/claim`).

Commit Message Conventions

- feat:** New feature (e.g. `feat: add Claim Reward functionality with UI and API integration`).
- fix:** Bug fix (e.g. `fix: Add suspect to case`).
- chore:** Build, tooling, dependencies, refactoring (e.g. `chore: add ESLint config`).
- doc:** Documentation changes (e.g. `doc: update README with API setup`).
- revert:** Revert a commit (e.g. `revert: revert feat: add payment`).

Format: `<type>: <short description>` (imperative mood, lowercase after colon).

Project Management

- Task creation:** Features and bugs are broken into small, testable tasks.
- Task distribution:** Assigned by role (frontend, backend, DevOps) or by feature area.
- Tracking:** GitHub Issues or similar for tasks and milestones.
- Documentation:** README and inline comments updated as features are added.
- Code review:** Pull requests before merging to main.

- **CI/CD:** Automated tests and build on push; Docker images built for deployment.

CI/CD Flow

The project uses **GitHub Actions** for CI/CD. The workflow runs on every push and pull request to `main`.

Triggers

- **Push** to `main`
- **Pull request** targeting `main`

Jobs (parallel, then sequential)

Job	Description
Lint	Runs ESLint; fails if there are lint errors.
Test	Runs Jest; fails if tests fail.
Build & Push	Runs only on push to <code>main</code> after lint and test pass; builds Docker image and pushes to GitHub Container Registry (GHCR).

Pipeline Steps

1. Lint job

- Checkout code
- Setup pnpm + Node 22
- Cache `node_modules` by `pnpm-lock.yaml` hash
- `pnpm install --frozen-lockfile`
- `pnpm lint`

2. Test job

- Same setup as Lint
- Cache Jest transforms
- `pnpm test`

3. Build & Push job (only on push to main)

- Depends on: Lint + Test
- Login to GHCR (`ghcr.io`)
- Build Docker image with `VITE_API_URL` from secrets
- Push image with tags: `latest` + git SHA
- Uses GitHub Actions cache for Docker layers

Required Secrets

Secret	Purpose
<code>GHCR_USERNAME</code>	GitHub username for GHCR login
<code>GHCR_TOKEN</code>	GitHub PAT or <code>GITHUB_TOKEN</code> for GHCR
<code>VITE_API_URL</code>	API base URL injected at build time

Local Docker

```
docker compose up --build
```

Builds the app with Vite, serves static files via nginx on port 80 (mapped to `APP_PORT` or 3000).

Key Entities

#	Entity	Purpose

1	User Entity	Represents authenticated police personnel; stores identity, role, and session for access control.
2	Role	Defines permissions (Administrator, Detective, Judge, etc.); used for role-based routing and guards.
3	Case	Central investigation unit; links crime, detective, evidence, suspects, and timeline.
4	Suspect	Person of interest; tracks status (suspected, wanted, arrested) and reward amount.
5	SuspectCrime	Links suspect to crime; supports case assignment and review workflow.
6	Evidence (BaseEvidence)	Base for all evidence types; ensures consistent fields (title, description, location, case).
7	Testimony	Witness statements with transcription and attachments; supports case narrative.
8	BiologicalEvidence	Physical evidence (images, lab results); used for forensic documentation.
9	VehicleEvidence	Vehicle-related evidence (model, color, plate); used for vehicle-related cases.
10	IdentificationEvidence	ID documents and owner info; used for identity verification.
11	CrimeScene	Location and context of a crime; links witnesses, examiner, and case report.
12	Complaint	Citizen complaints; workflow from cadet to officer approval and case assignment.
13	Reward	Monetary rewards for tips; tracks claim status and recipient.
14	Interrogation	Suspect questioning; supports scoring and sergeant review.

NPM Packages

Package	Purpose	Usage
@tanstack/react-query	Server state, caching, refetching	useQuery for API data (e.g. cases, employee count); replaces manual loading/error state.
react-hook-form + @hookform/resolvers + zod	Form state and validation	Login, register, evidence forms, payment; Zod schemas for validation.
zustand	Client state management	Auth session, detective board layout; persisted where needed.
@xyflow/react	Node-based diagrams	Detective board for visualizing evidence and case relationships.
axios	HTTP client	API calls with interceptors for JWT and token refresh.
react-router-dom	Routing	SPA routing, nested routes, RoleGuard for protected paths.
react-toastify	Toast notifications	Error/success feedback for API responses; shown via HTTP interceptor.
date-fns	Date formatting and manipulation	Format timestamps in case timeline, evidence, crime scenes.
@hugeicons/react + @hugeicons/core-free-icons	Icon library	Icons across UI (folder, user, timeline, car, fingerprint, etc.).
tailwindcss + tailwind-merge + clsx	Styling and class utilities	Utility-first CSS; cn() for conditional class merging in components.
html-to-image	Screenshot/export	Export detective board or other views to image.
radix-ui / shadcn	UI primitives and components	Accessible components (Dialog, Tabs, Select, Accordion); base for design system.
jest + @testing-library/react	Unit and component	Tests for UI components (Button, Badge, Input), stores (auth, detective-

Package	testing Purpose	board), schemas, utilities. Usage
---------	-----------------	--------------------------------------

AI-Generated Code Samples

1. HTTP Client with Token Refresh (src/lib/http.ts)

```
const refreshAccessToken = async (): Promise<string> => {
  const refresh = useAuthStore.getState().session?.refresh;
  if (!refresh) throw new Error("No refresh token");

  const { data } = await axios.post<{ access: string }>(
    `${API_URL}/auth/login/refresh/`,
    { refresh },
  );

  const session = useAuthStore.getState().session;
  if (session) {
    useAuthStore.getState().setSession({ ...session, access: data.access });
  }

  return data.access;
};
```

2. Evidence Node Component (src/components/detective-board/nodes/evidence-node.tsx)

```
const getIcon = (evidence: BaseEvidence) => {
  const e = evidence as unknown as Record<string, unknown>;
  if (e.transcription) return <HugeiconsIcon icon={ArchiveIcon} className="size-4" />;
  if (e.vehicle_model) return <HugeiconsIcon icon={Car01Icon} className="size-4" />;
  if (e.images) return <HugeiconsIcon icon={InjectionIcon} className="size-4" />;
  if (e.owner_first_name) return <HugeiconsIcon icon={FingerPrintIcon} className="size-4" />;
  return <HugeiconsIcon icon={ArchiveIcon} className="size-4" />;
};
```

3. Role Guard (src/guards/role.guard.tsx)

```
export default function RoleGuard({ allowedRoles }: { allowedRoles: readonly AllowedCaseRole[] }) {
  const session = useAuthStore((s) => s.session);

  if (!session) return <Navigate to="/login" replace />;
  if (!allowedRoles.includes(session.user.role_title)) return <Navigate to="/" replace />;

  return <Outlet />;
}
```

4. Detective Board Store with Persistence (src/stores/detective-board.store.ts)

```
export const useDetectiveBoardStore = create<DetectiveBoardState>(){
  persist(
    (set, get) => ({
      boards: {},
      updateBoard: (caseId, nodes, edges) =>
        set((state) => ({
          boards: { ...state.boards, [caseId]: { nodes, edges } },
        })),
      getBoard: (caseId) => get().boards[caseId],
    }),
    { name: "detective-board-storage" },
  ),
};
```

5. Zod Auth Schema (src/schemas/auth.schema.ts)

```
export const LoginSchema = z.object({
  identifier: z.string().min(1, "Username or Email is required"),
  password: z.string().min(1, "Password is required"),
});

export const RegisterSchema = z.object({
  username: z.string().min(3, "Username must be at least 3 characters"),
  email: z.string().email("Invalid email address"),
  password: z.string().min(8, "Password must be at least 8 characters"),
  first_name: z.string().optional(),
  national_id: z.string(),
  phone: z.string(),
  last_name: z.string().optional(),
});
```

AI Strengths & Weaknesses

Frontend Development

Strengths	Weaknesses
Fast scaffolding of components, pages, and forms	May over-engineer or add unnecessary abstractions
Good at React patterns (hooks, composition)	Can miss edge cases in forms and validation
Helpful for repetitive UI (tables, cards, modals)	May not match existing design system or conventions
Useful for TypeScript types from API schemas	Can produce verbose or redundant code
Effective for routing and layout structure	May not optimize for performance (memoization, lazy loading)
Good at integrating libraries (React Query, Zustand)	May not handle accessibility well
Works well on a solid base—extends and develops effectively when the initial architecture is good	Fails when the initial base is poor; quality of output depends heavily on project foundation
—	Tends to suggest outdated library versions, leading to compatibility issues and bugs

Requirements Analysis

Initial Requirements

- Police management system with case tracking
- User authentication and role-based access

- Case creation, assignment, and timeline
- Evidence management (multiple types)
- Crime scene reporting
- Complaint filing and review
- Suspect management and interrogation
- Reward system with payment
- Court/trial workflow for judges

Final Delivered Features

- JWT auth with refresh token and blacklist on logout
- Role-based routing (Administrator, Detective, Judge, etc.)
- Case CRUD, timeline, evidence, suspects
- Detective board (visual case mapping)
- Crime scene reporting with witnesses
- Complaint workflow (cadet → officer)
- Suspect add/review, interrogation with scoring
- Reward reports, claim flow, payment integration
- Admin panel for cases, roles, complaints, crime scenes, suspects, punishments

Decision Strengths

- **Zod + react-hook-form**: Strong validation and type safety.
- **Zustand**: Simple state, easy persistence for detective board.
- **React Query**: Caching and refetching without manual logic.
- **RoleGuard**: Centralized, declarative access control.
- **Docker + nginx**: Straightforward deployment.

Decision Weaknesses

- **Type assertions** (e.g. `as unknown as Record<string, unknown>`) in evidence nodes reduce type safety.
- **Monolithic App.tsx**: All routes in one file; could be split by domain.
- **Limited error boundaries**: Only root-level; more granular boundaries could improve UX.
- **No E2E tests**: Only unit tests; E2E would improve confidence in flows.

Getting Started

Installation

```
pnpm install
```

Environment

Copy `.env.example` to `.env` and set:

```
VITE_API_URL=http://localhost:8000/api
```

Development

```
pnpm dev
```

Build

```
pnpm build
```

Test

```
pnpm test
pnpm test:coverage
```

Docker

```
docker compose up --build
```

or use pull the image:

```
docker pull ghcr.io/l-a-noire/la-noire-frontend:latest
```

Project Structure

```
src/
├─ api/          # API client functions
├─ components/   # Reusable UI components
├─ guards/       # Route guards (e.g. RoleGuard)
├─ hooks/        # Custom hooks
├─ lib/          # Utilities (http, utils)
├─ pages/        # Page components
├─ providers/    # Context providers
├─ schemas/      # Zod validation schemas
├─ stores/       # Zustand stores
└─ types/        # TypeScript types
```